

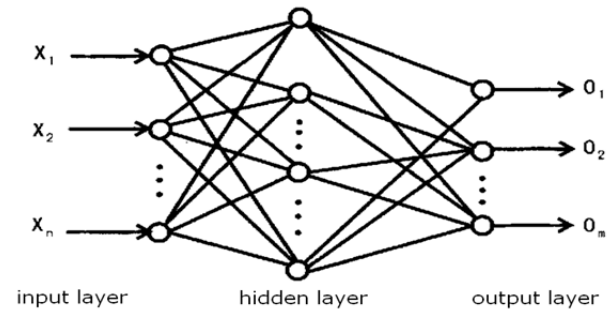
A Paper on BP Algorithm for Training MLP with Worked Examples

1 Introduction

A **multilayer perceptron** (MLP) is a feedforward artificial neural network model that maps sets of input data onto a set of appropriate outputs. An MLP consists of multiple layers of nodes in a directed graph, with each layer fully connected to the next one. Except for the input nodes, each node is a neuron (or processing element) with a nonlinear activation function. MLP utilizes a supervised learning technique called backpropagation for training the network. MLP is a modification of the standard linear perceptron and can distinguish data that are not linearly separable.

Backpropagation, an abbreviation for "backward propagation of errors", is a common method of training artificial neural networks. From a desired output, the network learns from many inputs, similar to the way a child learns to identify a dog from examples of dogs. It is a supervised learning method, and is a generalization of the delta rule. It requires a dataset of the desired output for many inputs, making up the training set. It is most useful for feed-forward networks (networks that have no feedback, or simply, that have no connections that loop). Backpropagation requires that the activation function used by the artificial neurons (or "nodes") be differentiable.

BP neural network structure is as follows:



The output of the neural is decided by the following

$$\text{formula: } Y = f\left(\sum_{i=1}^n X_i W_i - \theta\right).$$

According to the given problems, here provides a method of artificial neural network based on back-propagation algorithm implemented with MATLAB under windows system. The program can simulate a BP network with one hidden layer, whose number of neurons in 3 layers and other parameters of BP network can be changed. The principle, flow chart and source code are presented in the report.

2 Methods

For better understanding, the backpropagation learning algorithm can be divided into two phases: propagation and weight update.

Phase 1: Propagation

Each propagation involves the following steps:

1. Forward propagation of a training pattern's input through the neural network in order to generate the propagation's output activations.
2. Backward propagation of the propagation's output activations through the neural network using the training pattern target in order to generate the deltas of all output and hidden neurons.

Phase 2: Weight update

For each weight-synapse follow the following steps:

1. Multiply its output delta and input activation to get the gradient of the weight.
2. Subtract a ratio (percentage) of the gradient from the weight.

This ratio (percentage) influences the speed and quality of learning; it is called the *learning rate*. The greater the ratio, the faster the neuron trains; the lower the ratio, the more accurate the training is. The sign of the gradient of a weight indicates where the error is increasing, this is why the weight must be updated in the opposite direction. Repeat phase 1 and 2 until the performance of the network is satisfactory.

Task 1:

Simulate the function:

$$f_1(x) = \sin(x) \quad x \in [0, 2\pi]$$

with 9 training samples and 361 testing samples.

Task 2:

Simulate the function:

$$f_2(x) = |\sin(x)| \quad x \in [0, 2\pi]$$

with 9 training samples and 361 testing samples.

Task 3:

Simulate the function:

$$f_3(x) = \sin(x_1)/x_1 * \sin(x_2)/x_2 \quad x_1, x_2 \in [-10, 10]$$

with 11×11 training samples and 41×41 testing samples.

Task 4:

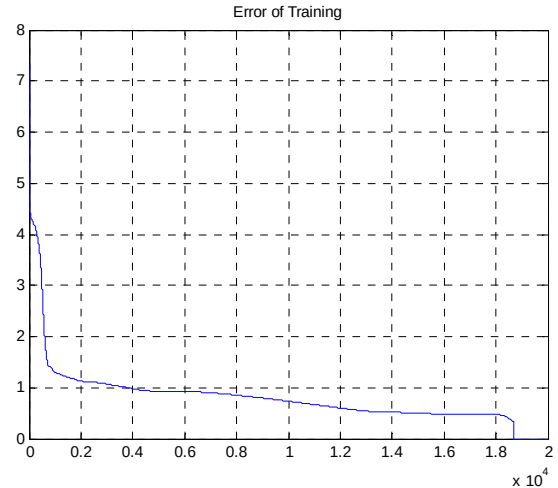
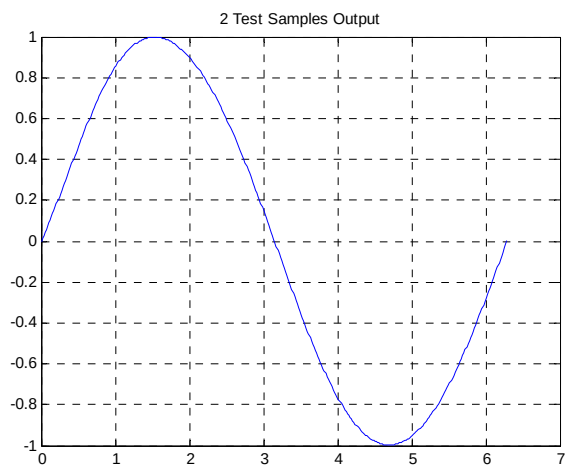
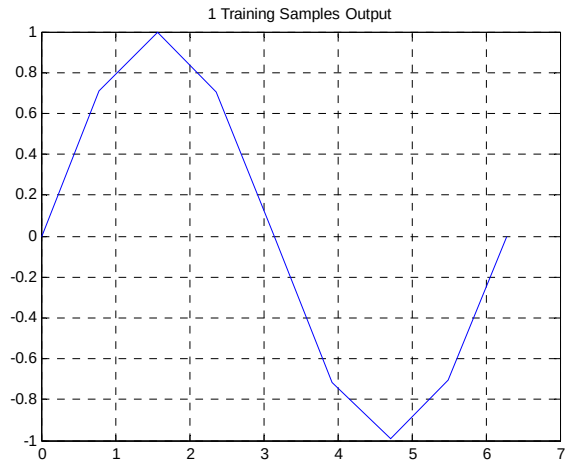
Simulate the function:

$$f_4(x) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2 \quad x_1, x_2 \in [-5, 5]$$

3 Results

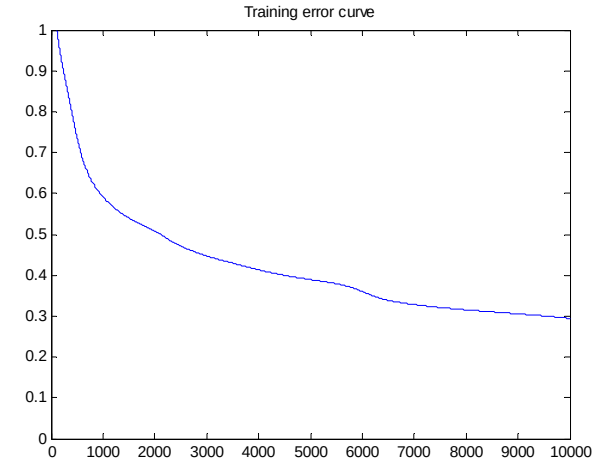
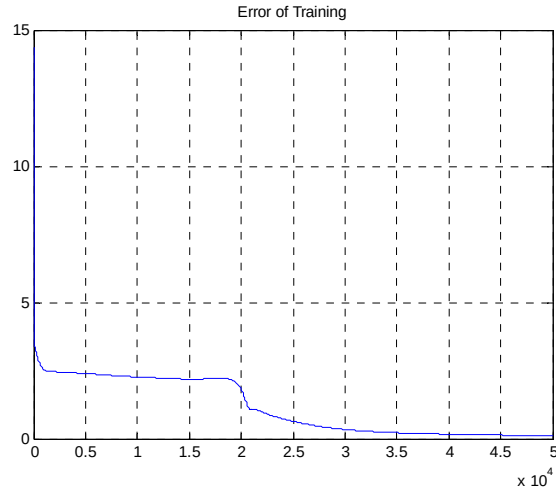
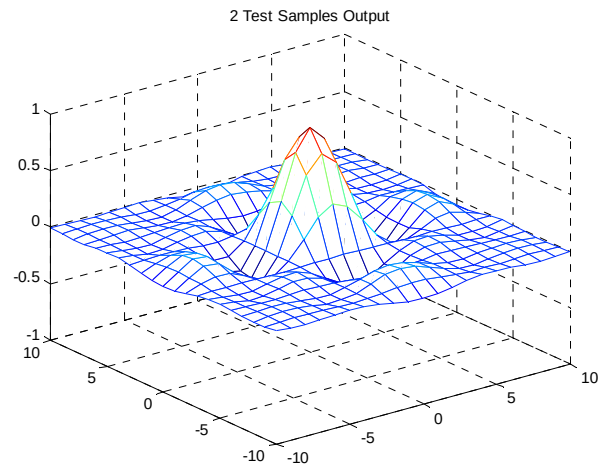
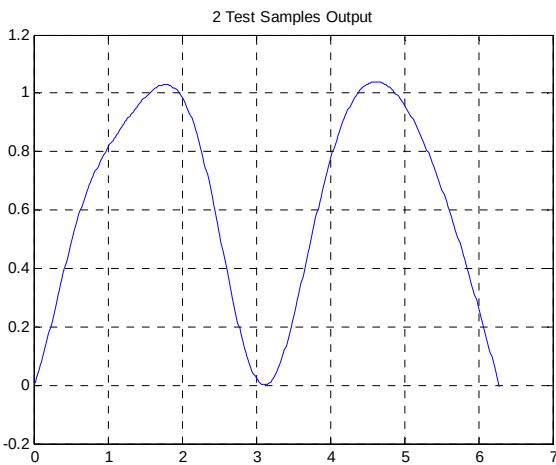
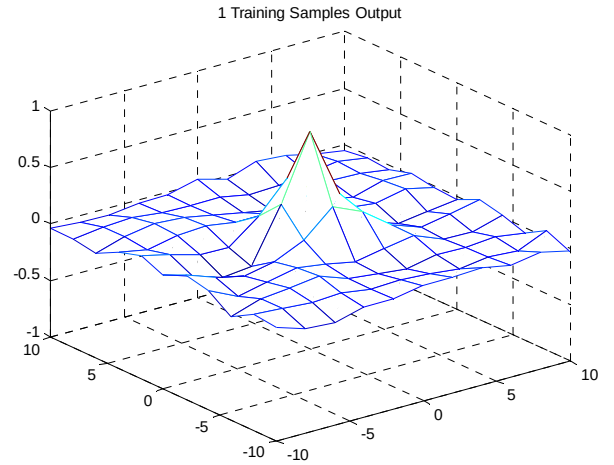
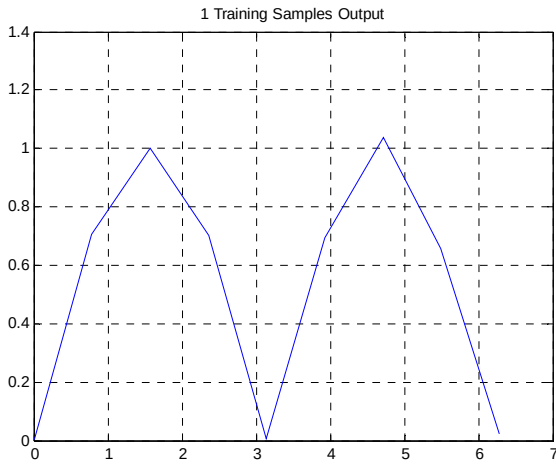
Task 1: $f_1(x) = \sin(x) \quad x \in [0, 2\pi]$

Let $\alpha=0.05$, $\eta=0.1$, time of training=20000 and number of neurons is 4.



Task 2: $f_2(x) = |\sin(x)| \quad x \in [0, 2\pi]$

Let $\alpha=0.05$, $\eta=0.1$, time of training=50000 and number of neurons is 4.

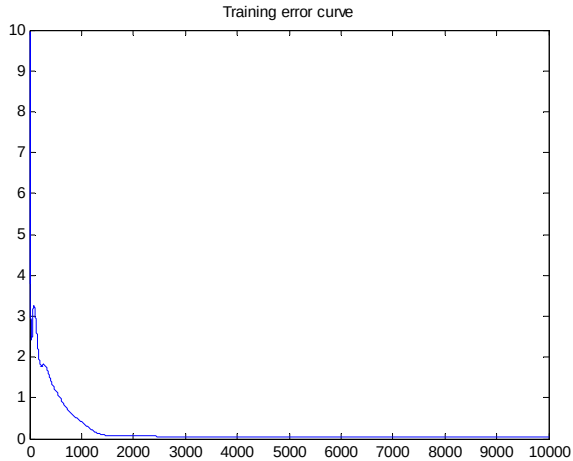
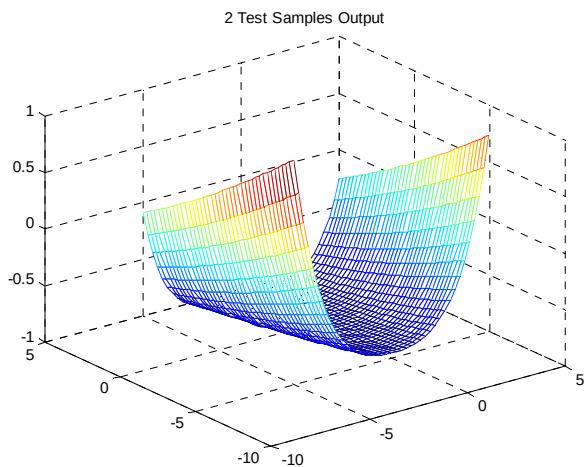
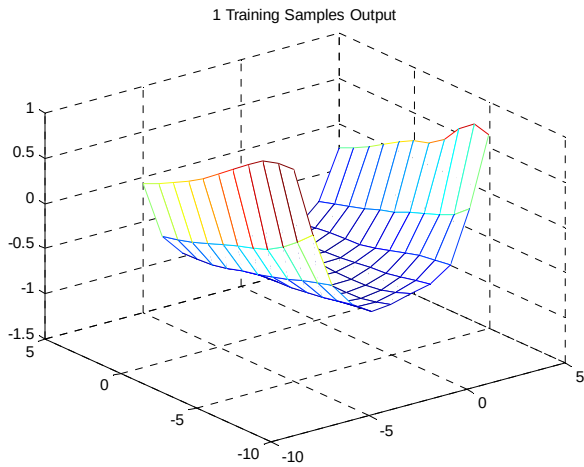


Task 3: $f_3(x) = \sin(x_1)/x_1 * \sin(x_2)/x_2$ $x_1, x_2 \in [-10, 10]$

Let $\alpha=0.01$, $\eta=0.1$, time of training=10000 and number of neurons is 20.

Task 4: $f_4(x) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$ $x_1, x_2 \in [-5, 5]$

Let $\alpha=0.1$, $\eta=0.5$, time of training=10000 and number of neurons is 10.



Appendix

Task 1:

clear all;

clc;

Ntr=9; % Number of Training Samples

Nts=361; % Number of Test Samples

Nhd=4; % Number of Hidden Layer Neurons

Nti=20000; % Time of Training

a=0.05; % Learning Rate

n=0.1; % Dynamic Learning Rate

Prec=0.001; % Default Precision

train=0:2*pi/(Ntr-1):2*pi; % Training Sample Input

expect=sin(train); % Desired Output

Thr=rand(Nhd,1); % Initial Threshold

Wih=[Thr,rand(Nhd,1)]; % Weight Matrix between Input Layer and Hidden Layer

Who=rand(1,Nhd); % Weight Matrix between Hidden Layer and Output Layer

PreWih=Wih; % Record last Weight Matrix between Input Layer and Hidden Layer

PreWho=Who; % Record last Weight Matrix between Hidden Layer and Output Layer

result=zeros(1,9); % Actual Output

error=zeros(1,Nti); % Training Error

count=0; % Time of Cycles

Jud=Prec+1; % Precision Judgment

while Jud>Prec

for i=1:Ntr % Forward calculation

input=[1,train(i)];

output=expect(i);

Layh_input=Wih*input'; % Input of Hidden Layer

Layh_output=ones(Nhd,1)./(ones(Nhd,1)+exp

(-Layh_input)); % Output of Hidden Layer

Layo=Who*Layh_output; % Value of Output Layer

result(1,i)=Layo; % Actual Output Value

Erro=(expect(i)-Layo); % Error of the Output Layer

Temp=exp(-Layh_input)./(ones(Nhd,1)+exp

(-Layh_input)).^2);

Errh=Temp.*(Erro*transpose(Who)); % Error of the

Hidden Layer

Temp=Wih+a*Errh*input+n*(Wih-PreWih);

PreWih=Wih;

Wih=Temp; %Update Weight Matrix between Input Layer and Hidden Layer

Temp=Who+a*Erro*transpose(Layh_output)+n*(Who -

PreWho);

```

        PreWho=Who;

        Who=Temp; %Update Weight Matrix between Hidden
Layer and Output Layer

    end

    count=count+1; % Time of Cycles

    for k=1:Ntr

        Jud=abs(result(1,k)-expect(k));

        error(count)=error(count)+abs(result(1,k)-expect(k)); %
Record the Training Error

    end

    if count==Nti

        break; % Jump out of the Loop when it reaches the Setting
Time

    end

end

```

```

test=0:2*pi/(Nts-1):2*pi; % Test Sample Input
expect=sin(test); %   Desired Output
output=zeros(1,Nts); % Actual Output
for i=1:Nts

    input=[1,test(i)];

    Layh_input=Wih*transpose(input);

    Layh_output=ones(Nhd,1)./(ones(Nhd,1)+exp(-Layh_input));

    Layo=Who*Layh_output;
output(1,i)=Layo; % Actual Output
end

```

```

figure;

plot(train,result);

grid on;

title('1 Training Samples Output')

```

```

figure;

plot(test,output);

grid on;

title('2 Test Samples Output');

figure;

plot(test,output, 'o',test,expect, '-');

```

```

grid on;

title('3 Contrast of Test and Ideal');

figure;

plot(test,expect-output);

grid on;

title('4 Error between Test and Ideal');

figure;

plot(1:Nti,error);

grid on;

title('Error of Training');

```

Task 2:

```

clear all;

clc;

Ntr=9; % Number of Training Samples
Nts=361; % Number of Test Samples
Nhd=8; % Number of Hidden Layer Neurons
Nti=50000; % Time of Training
a=0.05; % Learning Rate
n=0.1; % Dynamic Learning Rate
Prec=0.0001; % Default Precision
train=0:2*pi/(Ntr-1):2*pi; % Training Sample Input
expect=sin(train);

expect=abs(expect); % Desired Output
Thr=rand(Nhd,1); % Initial Threshold
Wih=[Thr,rand(Nhd,1)]; % Weight Matrix between Input Layer and
Hidden Layer
Who=rand(1,Nhd); % Weight Matrix between Hidden Layer and
Output Layer
PreWih=Wih; % Record last Weight Matrix between Input Layer and
Hidden Layer
PreWho=Who; % Record last Weight Matrix between Hidden Layer
and Output Layer
result=zeros(1,9); % Actual Output
error=zeros(1,Nti); % Training Error

count=0; % Time of Cycles

```

```

Jud=Prec+1; % Precision Judgment
while Jud>Prec
    for i=1:Ntr % Forward calculation
        input=[1,train(i)];
        output=expect(i);
        Layh_input=Wih*input'; % Input of Hidden Layer
        Layh_output=ones(Nhd,1)./(ones(Nhd,1)+exp
(-Layh_input)); % Output of Hidden Layer
        Layo=Who*Layh_output; % Value of Output Layer
        result(1,i)=Layo; % Actual Output Value
        Erro=(expect(i)-Layo); % Error of the Output Layer
        Temp=exp(-Layh_input)./((ones(Nhd,1)+exp
(-Layh_input)).^2);
        Errh=Temp.*(Erro*transpose(Who)); % Error of the
Hidden Layer
        Temp=Wih+a*Errh*input+n*(Wih-PreWih);
        PreWih=Wih;
        Wih=Temp; %Update Weight Matrix between Input Layer
and Hidden Layer
        Temp=Who+a*Erro*transpose(Layh_output)+n*(Who -
PreWho);
        PreWho=Who;
        Who=Temp; % Update Weight Matrix between Hidden
Layer and Output Layer
    end
    count=count+1; % Time of Cycles

    for k=1:Ntr
        Jud=abs(result(1,k)-expect(k));
        error(count)=error(count)+abs(result(1,k)-expect(k)); %
Record the Training Error
    end
    if count==Nti
        break; % Jump out of the Loop when it reaches the Setting
Time
    end
end

test=0:2*pi/(Nts-1):2*pi; % Test Sample Input
expect=(sin(test));

```

```

expect=abs(expect); % Desired Output
output=zeros(1,Nts); % Actual Output
for i=1:Nts
    input=[1,test(i)];
    Layh_input=Wih*transpose(input);
    Layh_output=ones(Nhd,1)./(ones(Nhd,1)+exp(-Layh_input));
    Layo=Who*Layh_output;
    output(1,i)=Layo; % Actual Output
end

figure;plot(train,result);grid;
title('1 Training Samples Output')

figure;plot(test,output);grid;
title('2 Test Samples Output');

figure;plot(test,output,'-',test,expect,'-');grid;
title('3 Contrast of Test and Ideal');

figure;plot(test,expect-output);grid;
title('4 Error between Test and Ideal');

figure;plot(1:Nti,error);grid;
title('Error of Training');

```

Task 3:

```

clear all;
clc;

Ntr=11; % Number of Training Samples Matrix
Nts=21; % Number of Test Samples Matrix
Nhd=20; % Number of Hidden Layer Neurons
Nti=10000; % Time of Training
a=0.01; % Learning Rate
n=0.1; % Dynamic Learning Rate
Prec=0.001; % Default Precision
train1=-10:(10-(-10))/(Ntr-1):10;
train2=-10:(10-(-10))/(Ntr-1):10; % Training Sample Input
[X1,X2]=meshgrid(train1-0.1,train2-0.1);
expect=sin(X1).*sin(X2)./X1./X2; % Desired Output

```

```

Thr1=rand(Nhd,1); % Initial Threshold for Hidden Layer
Thr2=rand(1,1); % Initial Threshold for Output Layer
Wih=rand(Nhd,2); % Weight Matrix between Input Layer and
Hidden Layer
Who=rand(1,Nhd); % Weight Matrix between Hidden Layer and
Output Layer
PreWih=Wih; % Last Weight Matrix between Input Layer and Hidden
Layer
PreWho=Who; % Last Weight Matrix between Hidden Layer and
Output Layer
PreThr1=Thr1; % Last Initial Threshold for Hidden Layer
PreThr2=Thr2; % Last Initial Threshold for Output Layer
Y=zeros(Ntr,Ntr); % Actual Output
error=zeros(1,Nti); % Training Error

count=0;
Jud=Prec+1;
while Jud>Prec
    Jud=0;
    for i=1:Ntr
        for j=1:Ntr
            % calculate the output of the neural network
            s1=[Thr1,Wih]*[1;train1(i);train2(j)];
            % the input of the hidden layer
            xo1=1./(1+exp(-s1));
            % the output of the hidden layer
            s2=[Thr2,Who]*[1;xo1];
            y=s2; %the output
            Y(j,i)=y;
            % calculate the error
            delta2=expect(j,i)-y; %the error of the output layer
            delta1=exp(-s1)./((1+exp(-s1)).^2).*(delta2*Who); % the
error of hidden layer
            % modify the value of weight
            Temp=Wih+a*delta1*[train1(i),train2(j)]+n*(Wih-PreWih);
            PreWih=Wih;
            Wih=Temp;
            Temp=Who+a*delta2*xo1'+n*(Who-PreWho);
            PreWho=Who;
            Who=Temp;
        end
    end
    % modify the value of threshold
    Temp=Thr1+a*delta1+n*(Thr1-PreThr1);
    PreThr1=Thr1;
    Thr1=Temp;
    Temp=Thr2+a*delta2+n*(Thr2-PreThr2);
    PreThr2=Thr2;
    Thr2=Temp;
    % calculate the value of the energy function
    Jud=Jud+0.5*(expect(j,i)-y).^2;
end
end
count=count+1; % Time of Cycles
error(count)=Jud;
if count==Nti
    break; % Jump out of the Loop when it reaches the Setting
end

Time
end

test1=-10.1:(9.9-(-10.1))/(Nts-1):9.9;
test2=-10.1:(9.9-(-10.1))/(Nts-1):9.9;
[T1,T2]=meshgrid(test1,test2);
except=sin(T1).*sin(T2)./T1./T2;

output=zeros(Nts,Nts);
for i=1:Nts
    for j=1:Nts
        s1=[Thr1,Wih]*[1;test1(i);test2(j)];
        % the input of the hidden layer
        train1=1./(1+exp(-s1)); % the output of the hidden layer
        s2=[Thr2,Who]*[1;train1];
        output(j,i)=s2; % output
    end
end
end

figure;
mesh(X1,X2,Y);
axis([-10 10 -10 10 -1 1])
title('1 Training Samples Output')

```

```
figure;
mesh(test1,test2,except)
axis([-10 10 -10 10 -1 1])
title('2 Test Samples Output')
```

```
figure;
mesh(test1,test2,output)
axis([-10 10 -10 10 -1 1])
title('3 Contrast of Test and Ideal')
```

```
figure;
mesh(test1,test2,except-output)
axis([-10 10 -10 10 -0.5 0.5])
title('4 Differences Between Test and Ideal')
```

```
figure;
plot(error);
axis([0 inf 0 1]);
title('Training error curve');
```

Task 4:

```
clear all;
clc;

Ntr=11; % Number of Training Samples Matrix
Nts=41; % Number of Test Samples Matrix
Nhd=10; % Number of Hidden Layer Neurons
Nti=10000; % Time of Training

a=0.1; % Learning Rate
n=0.5; % Dynamic Learning Rate
Prec=0.001; % Default Precision

train1=-5:(5-(-5))/(Ntr-1):5;
train2=-5:(5-(-5))/(Ntr-1):5; % Training Sample Input
[X1,X2]=meshgrid(train1-0.1,train2-0.1);
expect=100*(X2-X1.^2).^2+(1-X1).^2; % Desired Output
MA=max(max(expect));
MI=min(min(expect));
expect=(expect-(MA+MI)/2)*2/(MA-MI);
```

```
Thr1=rand(Nhd,1); % Initial Threshold for Hidden Layer
Thr2=rand(1,1); % Initial Threshold for Output Layer
Wih=rand(Nhd,2); % Weight Matrix between Input Layer and
Hidden Layer
Who=rand(1,Nhd); % Weight Matrix between Hidden Layer and
Output Layer

PreThr1=Thr1; % Last Initial Threshold for Hidden Layer
PreThr2=Thr2; % Last Initial Threshold for Output Layer
PreWih=Wih; % Last Weight Matrix between Input Layer and Hidden
Layer
PreWho=Who; % Last Weight Matrix between Hidden Layer and
Output Layer

Y=zeros(Ntr,Ntr); % Actual Output
error=zeros(1,Nti); % Training Error

for count=1:Nti % Time of Cycles
    Jud=0;
    for i=1:Ntr
        for j=1:Ntr
            % calculate the output of the neural network
            s1=[Thr1,Wih]*[1;train1(i);train2(j)];
            % the input of the hidden layer
            xo1=1./(1+exp(-s1));
            % the output of the hidden layer
            s2=[Thr2,Who]*[1;xo1];
            y=s2; %the output
            Y(j,i)=y;
            % calculate the error
            delta2=expect(j,i)-y; % the error of the output layer
            delta1=exp(-s1)./(1+exp(-s1)).^2.*(delta2*Who'); % the
error of hidden layer
            % modify the value of weight
            Temp=Wih+a*delta1*[train1(i),train2(j)]+n*(Wih-PreWih);
            PreWih=Wih;
            Wih=Temp;
            Temp=Who+a*delta2*xo1'+n*(Who-PreWho);
            PreWho=Who;
```



```

Who=Temp;

% modify the value of threshold

Temp=Thr1+a*delta1+n*(Thr1-PreThr1);

PreThr1=Thr1;

Thr1=Temp;

Temp=Thr2+a*delta2+n*(Thr2-PreThr2);

PreThr2=Thr2;

Thr2=Temp;

% calculate the value of the energy function

Jud=Jud+0.5*(expect(j,i)-y).^2;

end

end

error(count)=Jud;

if Jud<=Prec; break; end;

end

test1=-5.1:(4.9-(-5.1))/(Nts-1):4.9;
test2=-5.1:(4.9-(-5.1))/(Nts-1):4.9;
[T1,T2]=meshgrid(test1,test2);
except=100*(T2-T1.^2).^2+(1-T1).^2;
MA=max(max(except));
MI=min(min(except));
except=(except-(MA+MI)/2)*2/(MA-MI);

output=zeros(Nts,Nts);
for i=1:Nts
    for j=1:Nts
        s1=[Thr1,Wih]*[1;test1(i);test2(j)];
        % the input of the hidden layer
        train1=1./(1+exp(-s1)); % the output of the hidden layer
        s2=[Thr2,Who]*[1;train1];
        output(j,i)=s2; % output
    end
end

figure;
mesh(X1,X2,Y);
title('1 Training Samples Output');

figure;

mesh(test1,test2,except);
title('2 Test Samples Output');

figure;
mesh(test1,test2,output);
title('3 Contrast of Test and Ideal');

figure;
mesh(test1,test2,except-output);
title('4 Differences Between Test and Ideal');

figure;
plot(error);
title('Training error curve');

```